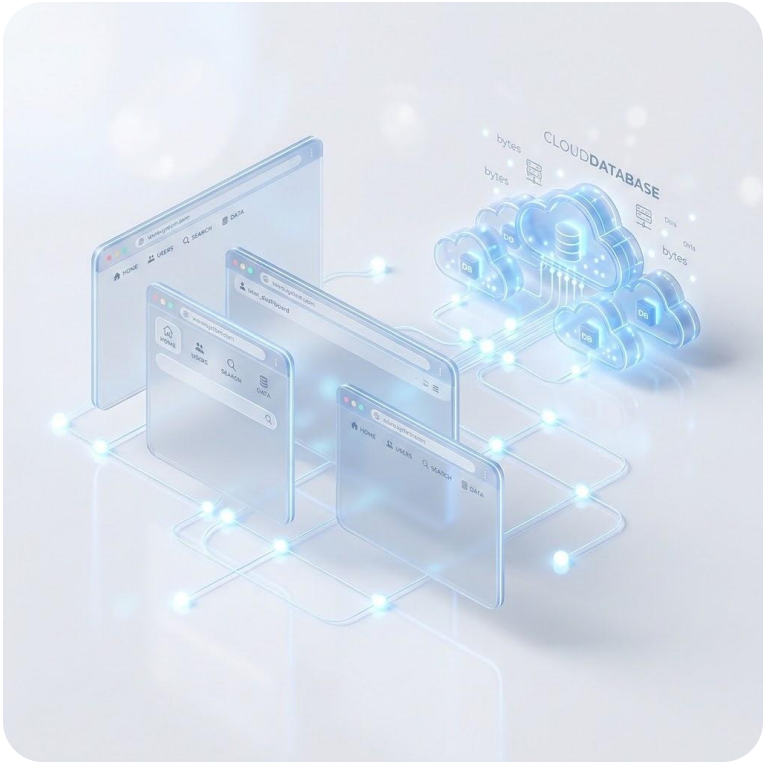
Departamento de Informática
UFV Universidade Federal de Viçosa

INF 321

Projeto e Desenvolvimento de Sistemas para a Web

Unidade 2.1:
Desenvolvimento Front-End |> HTML

Prof. Lucas Vegi



HTML

HyperText Markup Language



A Linguagem da Web

A linguagem de marcação padrão da Web, essencial para construir e estruturar páginas e documentos online.

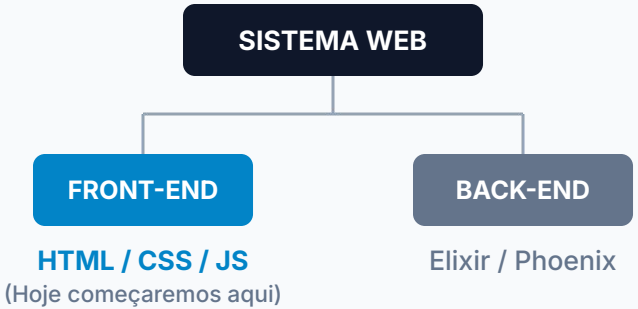


Estrutura & Significado

Define o conteúdo e a estrutura de uma página Web, atribuindo propósito e significado semântico a cada elemento visual.

Onde HTML se encaixa?

O Ecossistema Web



Componentes do Front-End

HTML | estrutura
Define o esqueleto, seções e conteúdo semântico da página.

CSS | aparência
Controla as cores, fontes, espaçamentos e design visual.

JavaScript | comportamento
Adiciona dinamismo, interatividade e funções complexas.

O que é HTML?

Conceito Principal

HTML (HyperText Markup Language) é a linguagem de marcação padrão utilizada para estruturar documentos e páginas na Web.

Ela utiliza elementos (tags) para descrever e organizar o conteúdo, permitindo que o navegador interprete essa marcação e renderize a página final.

Exemplo de Marcação

```
<h1>Meu site</h1>
<p>Este é um parágrafo. </p>
```



Importante

HTML não é uma linguagem de programação.
Por ser uma linguagem de marcação, ela não possui lógica condicional, loops ou processamento de dados.

HTML não é programação

HTML | estrutura

```
<h1>Produtos</h1>
<p>Notebook</p>
```

Apenas descreve e organiza o conteúdo:
"Existe um título e um parágrafo."

JavaScript | programa

```
if (price > 1000) {
  discount = 10;
}
```

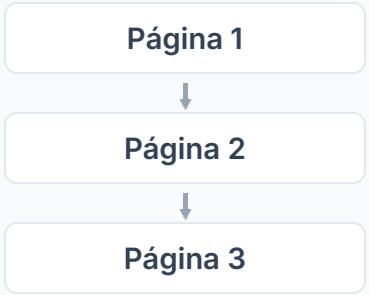
Instrui ações e toma decisões computacionais:

- **Lógica** e tomada de decisões
- **Condições** dinâmicas
- **Processamento** ativo de dados

O que significa HyperText?

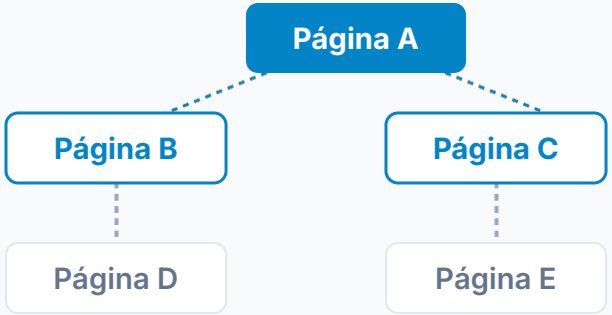
Texto Tradicional

Estrutura linear, sequencial e estática



Hipertexto

Estrutura multidimensional e não-linear



O usuário pode navegar livremente entre os documentos através de **links**.

Elementos HTML

HTML é composto por elementos. Veja a anatomia e estrutura de um elemento padrão:

1. Abertura

`<p>`

Indica o início do elemento e define o seu tipo (neste caso, um parágrafo).

2. Conteúdo

Olá, mundo!

A informação ou texto que será renderizado pelo navegador e exibido ao usuário.

3. Fechamento

`</p>`

Indica o término do elemento. É idêntica à abertura, mas possui uma barra inclinada.

O elemento completo estruturado: `<p>Olá, mundo!</p>`

Elementos sem conteúdo

Nem todos os elementos possuem fechamento. Veja exemplos de elementos que não contêm conteúdo:

Elemento <img>

```
<img src="foto.jpg" alt="...">
```

Define uma imagem na página. Não possui tag de fechamento por não conter texto interno; seus atributos fornecem toda a informação necessária.

Elemento

```
<br>
```

Insere uma quebra de linha no texto. Trata-se de uma ação direta na estrutura visual, dispensando qualquer tag de fechamento.

Esses elementos são chamados de **elementos vazios (void elements)** ou sem conteúdo.

Aninhamento

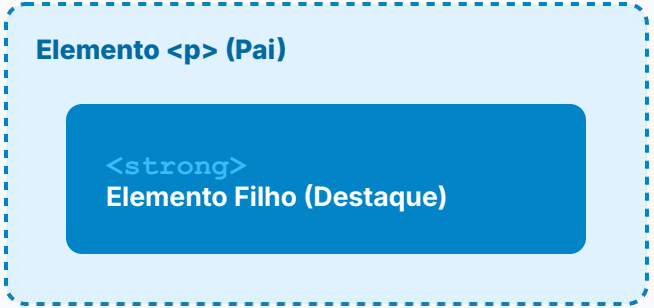
Elementos podem estar dentro de outros elementos:

Código Exemplo

```
<p>
Este é um texto com
<strong>destaque</strong>.
</p>
```

Este é um texto com **destaque**.

Visualização da Estrutura



O HTML possui uma estrutura essencialmente hierárquica e em árvore.

Comentários em HTML

Comentários servem para documentar e organizar o código sem alterar a exibição da página.

```
<!-- Sintaxe de um comentário -->
<!-- Exemplo de uso -->
<body>
  <!-- Cabeçalho da seção -->
  <h1>Minha loja</h1>
</body>
```

Para que utilizar?

- Documentar trechos específicos do código.
- Facilitar a organização e futura manutenção.
- Explicar decisões de design ou lógica complexa.
- Auxiliar e desativar blocos temporariamente durante o desenvolvimento.

Atenção: Os comentários são enviados ao navegador. Nunca coloque informações sensíveis neles (como senhas ou chaves de API).

Estrutura básica

Todo documento HTML moderno pode começar com esta estrutura:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Minha página</title>
  </head>
  <body>
    <h1>Olá, mundo!</h1>
  </body>
</html>
```

`<head>` (Cabeçalho)

Contém os metadados do documento, como o título da aba do navegador, links para arquivos CSS e scripts.

`<body>` (Corpo)

Engloba todo o conteúdo visível da página que é renderizado pelo navegador, como textos, títulos, imagens e links.

→ Vamos entender em detalhes o papel de cada uma dessas partes.

DOCTYPE

Instrução obrigatória que deve aparecer no início de qualquer documento HTML moderno.

```
<!DOCTYPE html>
```

Propósito da instrução

- Indica explicitamente que o documento está utilizando o padrão **HTML moderno (HTML5)**.
- Garante que o navegador interprete e renderize o documento no **modo esperado (padrão)**, evitando falhas de layout.

Embora pareça uma tag, o **<!DOCTYPE>** é apenas um preâmbulo necessário para o navegador.

Elemento `<html>`

É o elemento raiz do documento. Tudo relacionado ao documento HTML fica dentro dele.

```html
<html lang="pt-BR">
...
</html>
```

Por que indicar o idioma?

- Promove a **acessibilidade** na web.
- Permite o funcionamento correto de **leitores de tela**.
- Facilita a indexação por **mecanismos de busca** (SEO).
- Garante a **interpretação correta** do idioma pelo navegador.

O atributo **lang="pt-BR"** define o português do Brasil como o idioma principal do documento.

Elementos `<head>` e `<body>`

Um documento possui duas regiões fundamentais com propósitos distintos:

```
<html>
  <head>
    <!-- informações sobre o documento -->
  </head>
  <body>
    <!-- conteúdo da página -->
  </body>
</html>
```

`<head>` (Cabeçalho)

Contém metadados, recursos essenciais e instruções sobre o documento que não aparecem de forma visível na página.

`<body>` (Corpo)

Engloba todo o conteúdo visível que é renderizado pelo navegador para o usuário, como textos, mídias e componentes interativos.

→ Ambas as áreas são estruturadas juntas para dar vida a qualquer página na web.

Elemento `<head>`

Define o cabeçalho do documento, guardando metadados e configurações cruciais para o navegador.

```html
<head>
  <meta charset="UTF-8">

  <meta name="viewport"
      content="width=device-width...">

  <title>Minha página</title>
</head>
```

O que podemos encontrar?

- **Título da página** (<title>)
- **Metadados** e codificação (UTF-8)
- **Configuração de responsividade** (viewport)
- **Links para estilos CSS externos**
- **Scripts JavaScript** e integrações
- **Outras informações** para SEO

As informações contidas no **<head>** preparam o navegador para renderizar o corpo da página corretamente.

Favicon – Ícone da página

O favicon é o pequeno ícone exibido na aba do navegador, favoritos e histórico.

```html
<head>
  <!-- Associa o ícone -->
  <link rel="icon" href="favicon.png">
  <!-- Também suporta SVG -->
  <link rel="icon" href="favicon.svg"
  type="image/svg+xml">
</head>
```

1. Organize o arquivo

Salve o ícone na raiz do projeto:

```
meu-site/
├── index.html
└── favicon.png (ou .svg)
```

2. Adicione no <head>

O elemento **<link>** com **rel="icon"** informa ao navegador.

Dê preferência ao formato SVG pela escalabilidade.

O favicon é um **metadado** . Por isso, a tag `<link>` deve sempre ficar dentro do cabeçalho `<head>`.

Elemento `<title>`

Define o título do documento, exibido na aba do navegador, favoritos e resultados de busca.

```html
<head>
  <title>Produtos</title>
</head>
```

Não confundir!

- **<title>** pertence aos metadados e define o título oficial do documento no navegador.
- **<h1>** faz parte do conteúdo da página, sendo o título principal visível na tela.

O `<title>` é fundamental para indexação e SEO, enquanto o `<h1>` estrutura a hierarquia visual do conteúdo.

Títulos e Hierarquia

O HTML possui seis níveis de títulos para estruturar a importância e a hierarquia do conteúdo.

```html
<h1>Título principal</h1>
<h2>Seção</h2>
<h3>Subseção</h3>
<h4>Subsubseção</h4>
<h5>Título menor</h5>
<h6>Título mínimo</h6>
```

Hierarquia e Estrutura

- **h1** é o título principal da página (recomenda-se apenas um por página).
- **h2 e h3** subdividem o conteúdo em seções e subseções lógicas.
- **h4 a h6** são utilizados para detalhamentos profundos.
- **Acessibilidade:** Leitores de tela utilizam títulos para navegar pela página.

Use as tags de título estritamente para marcar a **hierarquia semântica** , e nunca apenas para mudar o tamanho do texto.

Elemento

Define parágrafos de texto, aplicando espaçamento vertical automático antes e depois do bloco.

```
<!-- Parágrafo simples -->
<p>Este é um parágrafo.</p>

<!-- Exemplo com título -->
<h1>Engenharia de Software</h1>
<p>
  Engenharia de Software envolve métodos,
  técnicas e ferramentas para sistemas.
</p>
```

Características do

- **Elemento de bloco:** Sempre inicia em uma nova linha e ocupa toda a largura disponível.
- **Margens automáticas:** Os navegadores adicionam uma margem superior e inferior por padrão.
- **Quebra de linha interna:** Espaços em branco e quebras consecutivas no código são ignorados.

Evite usar parágrafos vazios apenas para criar espaçamento visual. Prefira usar propriedades CSS.

Formatação semântica

A ideia é preferir elementos que expressem significado, e não apenas aparência.

```
<!-- Destacando conteúdo -->
<strong>Importante</strong>
<em>ênfase</em>
<!-- Também podemos utilizar -->
<mark>texto destacado</mark>
<del>Importante</del>
```

Estrutura Semântica

- **** dá relevância e seriedade ao conteúdo, aplicando negrito por padrão.
- **** confere ênfase tonal ao trecho selecionado, renderizado em itálico.
- **<mark>** destaca trechos de alta relevância para fins de referência rápida.
- **<del>** indica um trecho que foi deletado ou editado em um documento.

Evite usar tags puramente estéticas como <b> ou <i>. O HTML semântico melhora a acessibilidade e indexação por mecanismos de busca.

Elemento <a>

O principal elemento da Web, utilizado para criar hiperlinks e conectar diferentes recursos.

```
<!-- Criando um hiperlink -->
<a href="https://www.ufv.br">
   UFV
</a>
```

Anatomia do Hiperlink

- **<a>:** Elemento "âncora" que inicia e encerra a marcação do link.
- **href:** Atributo essencial que define o destino (URL ou caminho).
- **Valor ("https://..."):** O endereço real para onde o navegador direcionará.
- **Conteúdo (UFV):** O texto visível e clicável na página pelo usuário.

Ao ser clicado, o texto âncora dispara a navegação ativa do usuário em direção ao recurso apontado pelo atributo `href`.

Links internos e caminhos

Podemos utilizar caminhos relativos ou absolutos para estruturar a navegação.

```html
<!-- Caminho relativo -->
<a href="/produtos">Produtos</a>

<!-- Caminho absoluto -->
<a href="https://exemplo.com.br/produtos">
  Produtos
</a>
```

Caminho Absoluto vs. Relativo

- **Caminho Relativo:** Aponta para um recurso dentro do próprio domínio (ex: /produtos). Se o site for exemplo.com.br, vira exemplo.com.br/produtos.
- **Caminho Absoluto:** Contém a URL completa do recurso, incluindo obrigatoriamente o protocolo e o domínio (ex: https://exemplo.com.br/produtos).

O atributo `href` define o endereço do recurso de destino para o qual o hiperlink aponta.

Link → HTTP

Como um clique do usuário aciona a comunicação e as requisições na Web.

```
<!-- Ao clicar no link -->
<a href="/produtos">Produtos</a>

1. Clique no link (Navegador)
   ↓ Dispara a requisição
2. GET /produtos HTTP/1.1
      Host: www.exemplo.com.br
   ↓ Processado pelo
3. Servidor Web → Resposta HTTP
```

Relação com o que já estudamos

HTML (Hipertexto)

`<a href="/produtos">`

↓ inicia o fluxo de navegação

HTTP (Protocolo Web)

`GET /produtos`

↓ enviado e respondido pelo

Servidor Web (Hospedagem)

Um link HTML pode iniciar uma requisição HTTP para obter e exibir outro recurso na Web.

Onde abrir o link?

O atributo `target` define em qual aba ou janela o recurso de destino será aberto.

```html
<!-- Abrir na mesma aba -->
<a href="/produtos"
target="_self">Produtos</a>

<!-- Abrir em nova aba -->
<a href="https://ufv.br"
target="_blank">UFV</a>
```

Valores comuns do target

- `_self:` Mesma aba/janela (comportamento padrão)
- `_blank:` Nova aba/janela
- `_parent:` Contexto pai (framesets)
- `_top:` Contexto superior completo

O atributo `_self` é o comportamento padrão, enquanto o `_blank` é amplamente utilizado em links externos para manter o usuário no site original.

Elemento `<img>`

Utilizado para incorporar de forma direta recursos de imagens e ilustrações em páginas Web.

```html
<!-- Inserindo uma imagem -->
<img
  src="notebook.jpg"
  alt="Notebook sobre uma mesa"
/>
```

Anatomia da Imagem

- **src:** Atributo obrigatório que define o caminho (URL local ou externa) onde o arquivo de imagem está hospedado.
- **alt:** Texto descritivo alternativo. É exibido caso a imagem falhe ao carregar ou para leitores de tela em termos de acessibilidade.

O preenchimento do atributo `alt` é um requisito indispensável para garantir a plena acessibilidade de pessoas com deficiência visual.

Por que usar `alt`?

Imagine um usuário com deficiência visual utilizando um leitor de tela para navegar.

```
<!-- Sem o atributo alt -->
<img src="notebook.jpg"/>
↳ Leitor diz: "imagem" ou "notebook.jpg"

<!-- Com o atributo alt -->
<img src="notebook.jpg" alt="Notebook sobre
uma mesa"/>
↳ Leitor diz: "Notebook sobre uma mesa"
```

O Impacto da Acessibilidade

- **Compreensão de Conteúdo:** Sem a descrição textual, o usuário perde totalmente o contexto e o significado do elemento visual.
- **Semântica muito além da Tela:** O HTML não serve apenas para "fazer aparecer coisas". Ele descreve o real significado e valor da informação.

O preenchimento correto do atributo `alt` é um pilar indispensável para construir uma Web verdadeiramente inclusiva.

WCAG (Web Content Accessibility Guidelines, ou Diretrizes de Acessibilidade para Conteúdo Web) - versão 3.0

Listas não ordenadas

Utilizamos `<ul>` *(unordered list)* para criar uma lista e `<li>` *(list item)* para cada item.

```html
<!-- Código HTML -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Resultado no Navegador

- HTML
- CSS
- JavaScript

A ordem dos elementos não é importante na exibição de uma lista não ordenada.

Listas ordenadas

Quando a **ordem dos itens é importante**, utilizamos a tag `<ol>` *(ordered list)*.

```html
<!-- Código HTML -->
<ol>
  <li>Instalar o Elixir</li>
  <li>Criar o projeto</li>
  <li>Executar a aplicação</li>
</ol>
```

Resultado no Navegador

1. Instalar o Elixir
2. Criar o projeto
3. Executar a aplicação

Comparando: `<ul>` gera marcadores não ordenados (• HTML), enquanto `<ol>` numera sequencialmente (1. HTML).

Sublistas hierárquicas

Uma lista pode conter outra lista. A sublista deve estar dentro do `<li>` ao qual pertence.

```html
<!-- Sublistas em HTML -->
<ul>
  <li>Front-end
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
</ul>
```

Resultado no Navegador

- **Front-end**
  - HTML
  - CSS

Listas HTML podem representar estruturas hierárquicas de vários níveis complexos.

Tabelas

HTML também representa dados tabulares.

```html
<!-- Código HTML -->
<table>
   <tr>
      <th>Produto</th>
      <th>Preço</th>
   </tr>
   <tr>
      <td>Notebook</td>
      <td>R$ 5.000</td>
   </tr>
</table>
```

Resultad o no Navegador

Produto	Preço
Notebook	R$ 5.000

Elementos: `<table>` (tabela), `<tr>` (linha), `<th>` (cabeçalho) e `<td>` (célula).

Mesclando colunas

O atributo **`colspan`** faz uma célula ocupar duas ou mais colunas.

```html
<!-- Código HTML -->
<table>
  <tr>
    <th colspan="2">Produto</th>
  </tr>
  <tr>
    <td>Notebook</td>
    <td>R$ 5.000</td>
  </tr>
</table>
```

Resultado no Navegador

Produto	
Notebook	R$ 5.000

O atributo `colspan="2"` faz a célula do cabeçalho (<th>) mesclar e ocupar 2 colunas.

Mesclando linhas

O atributo **rowspan** faz uma célula ocupar duas ou mais linhas.

```html
<!-- Código HTML -->
<table>
  <tr>
    <th>Categoria</th>
    <th>Produto</th>
  </tr>
  <tr>
    <td rowspan="2">Notebook</td>
    <td>Modelo A</td>
  </tr>
  <tr>
    <td>Modelo B</td>
  </tr>
</table>
```

Resultado no Navegador

Categoria	Produto
Notebook	Modelo A
	Modelo B

O atributo `rowspan="2"` faz a célula de dados (<td>) mesclar e ocupar 2 linhas sequenciais.

Formulários

Formulários são fundamentais em aplicações Web.

```html
<form>
  <label>Nome:</label>
  <input type="text">
  <label>Email:</label>
  <input type="email">
  <button>Enviar</button>
</form>
```

Resultado no Navegador

Nome:

Email:

Enviar

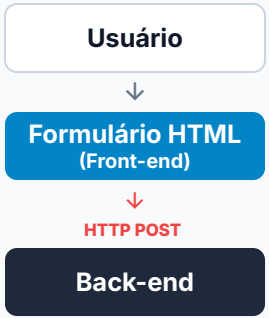
O usuário fornece dados. Posteriormente veremos como esses dados chegam ao **back-end**.

Formulários e HTTP

Um formulário pode enviar dados para o servidor.

```html
<!-- Envio via POST -->
<form method="post" action="/products">
  <label>Nome:</label>
  <input type="text" name="nome">
  <button>Enviar</button>
</form>
```

Fluxo de Comunicação



1. Entrada: O usuário preenche e interage com os campos.

2. Transporte: Os dados são enviados via método HTTP POST.

3. Destino: O back-end recebe e processa na rota definida.

⇄ **HTML → HTTP → Back-end:** Isso conecta diretamente a estrutura da página ao servidor web.

GET e POST em formulários

O método de envio define como os dados do formulário serão transmitidos ao servidor.

```
<!-- Envio via GET -->
<form method="get" action="/search">
```

Método GET (obter/consultar)

Anexa os dados diretamente na URL da requisição. Recomendado para buscas e consultas que não provocam alterações no servidor.

```
<!-- Envio via POST -->
<form method="post" action="/products">
```

Método POST (enviar/criar)

Envia os dados no corpo da requisição HTTP. Essencial para criar recursos, fazer uploads ou enviar informações sensíveis.

Isso retoma as bases e conceitos que estudamos detalhadamente na nossa aula de HTTP.

Exemplo de GET em formulário

Como os dados preenchidos são mapeados e transmitidos na URL.

```
<!-- Um único campo de busca -->
<form action="/produtos" method="GET">
  <input type="text" name="busca">
</form>
```

Requisição Gerada (URL):

`/produtos?busca=notebook`

- **?** inicia os parâmetros da consulta
- **busca** é o nome do parâmetro (campo)
- **notebook** é o valor digitado pelo usuário

```
<!-- Múltiplos campos no formulário -->
<form action="/produtos" method="GET">
  <input name="busca"> <input name="preco_max">
</form>
```

Com Vários Campos:

`/produtos?busca=notebook&preco_max=5000`

- **&** separa as chaves de parâmetros adicionais
- Gera pares **chave=valor** na URL
- Fácil de compartilhar e salvar nos favoritos
- **notebook** e **5000** são valores digitados pelo usuário

No método GET, os dados do formulário viram parâmetros da URL, tornando as consultas fáceis de compartilhar e favoritar.

Controles de Formulário

Além de campos de texto, HTML possui diferentes controles para entrada de dados.

Radio button — uma opção

- (●) Ciência da Computação
- ( ) Sistemas de Informação

Apenas uma opção pode estar selecionada de cada vez dentro de um mesmo grupo de botões.

```html
<!-- Grupo de Opções -->
<label>
  <input type="radio" name="curso" value="cc">
  Ciência da Computação
</label>
<label>
  <input type="radio" name="curso" value="si">
  Sistemas de Informação
</label>
```

O mesmo atributo `name` garante que o usuário possa selecionar **apenas uma opção** .

Controles de Formulário

Além de campos de texto, HTML possui diferentes controles para entrada de dados.

Checkbox — múltiplas opções

- [x] HTML
- [x] CSS
- [ ] JavaScript

Cada checkbox funciona de forma independente, permitindo marcar ou desmarcar livremente.

```
<!-- Múltiplas Opções -->
<label>
  <input type="checkbox" name="tecnologias"
  value="html" checked>
  HTML
</label>
<label>
  <input type="checkbox" name="tecnologias"
  value="css" checked>
  CSS
</label>
...
```

O usuário pode selecionar **zero, uma ou várias opções** simultaneamente dentro do grupo.

Controles de Formulário

Além de campos de texto, HTML possui diferentes controles para entrada de dados.

Combo box — `<select>`

Curso:

Ciência da Computação

Radio button → escolher UMA opção
Checkbox → escolher VÁRIAS opções
Select → escolher em uma LISTA

```
<!-- Combo Box / Seleção -->
<label for="curso">Curso:</label>
<select id="curso" name="curso">
  <option value="cc">Ciência da Computação</option>
  <option value="si">Sistemas de Informação</option>
</select>
```

Todos esses controles podem enviar dados para o back-end quando fazem parte de um `<form>`.

HTML semântico

HTML não é apenas aparência. É sobre o significado do conteúdo.

Como Pensar?

Evite pensar:
"Qual tag deixa isso bonito?"

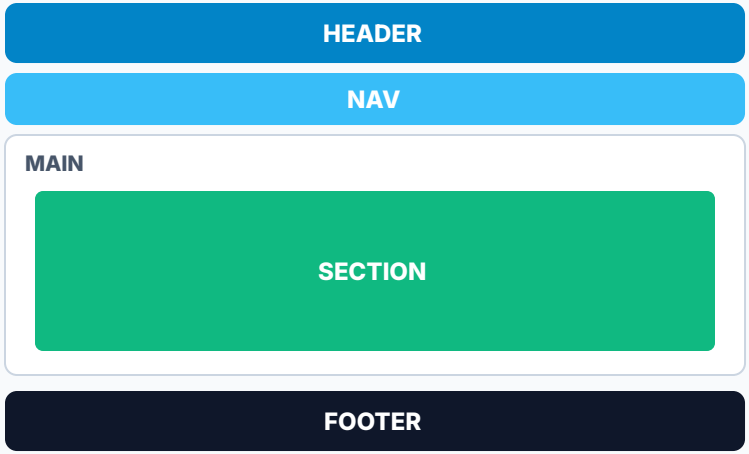
Prefira pensar:
"O que esse conteúdo significa?"

```
<!-- Estrutura Semântica -->
<header>
  <nav>...</nav>
</header>
<main>
  <section>
    <article>...</article>
  </section>
  <aside>...</aside>
</main>
<footer>...</footer>
```

Esses elementos descrevem de forma precisa a estrutura semântica da página para navegadores e leitores de tela.

Estrutura semântica

Uma página pode ser organizada assim:



A estrutura ajuda:

- Usuários
- Leitores de tela
- Mecanismos de busca
- Desenvolvedores

O HTML semântico indica de forma precisa o significado do conteúdo para navegadores, robôs e tecnologias assistivas.

e

Dois elementos genéricos e sem significado semântico específico muito utilizados em HTML.

```
<!-- Conteúdo em Bloco -->
<div>
  <p>Notebook</p>
</div>
```

Elemento

É normalmente utilizado como contêiner de conteúdo em bloco para agrupar elementos estruturais e aplicar estilos ou layouts a seções inteiras da página.

```
<!-- Conteúdo em Linha -->
<p>
  Preço: <span>R$ 5.000</span>
</p>
```

Elemento

É normalmente utilizado para agrupar, destacar ou identificar um trecho específico de conteúdo ou formatação diretamente dentro de uma linha de texto.

Pense no `div` como um contêiner maior (bloco) e no `span` como um contêiner menor dentro do texto.

Exemplos úteis de `<div>`

Como e quando utilizar o elemento `div` como contêiner na prática.

Agrupando Componentes

```html
<!-- Agrupando uma estrutura de produto -->
<div class="produto">
  <h2>Notebook</h2>
  <p>R$ 5.000</p>
  <button>Comprar</button>
</div>
```

Uso como Contêiner:

Permite estruturar e agrupar elementos que pertencem à mesma interface. Posteriormente, a classe `.produto` será utilizada para aplicar estilos de forma unificada no CSS.

Agrupando Áreas de Menu

```html
<!-- Agrupando conjunto de links -->
<div class="menu">
  <a href="/">Início</a>
  <a href="/produtos">Produtos</a>
  <a href="/contato">Contato</a>
</div>
```

Organização Semântica:

Ideal para agrupar e organizar áreas específicas da página quando não há uma tag semântica (como `<nav>` ou `<aside>`) mais adequada disponível.

O `<div>` é extremamente útil para agrupar elementos, mas utilize-o apenas quando não existir um elemento semântico mais apropriado.

Exemplos úteis de `<span>`

Como aplicar o elemento `span` para identificar e destacar trechos de texto na prática.

Destacando e Estilizando

```
<!-- Destacando partes de um texto -->
O produto custa <span class="preco">R$ 5.000</span>.
Status: <span class="disponivel">Disponível</span>.
Este produto está <span class="destaque">em promoção</span>.
```

Identificação Precisa:

O **span** permite isolar trechos específicos de texto para que, posteriormente, classes CSS como **.preco**, **.disponivel** ou **.destaque** definam sua aparência.

Diferença Estrutural

```
<!-- div vs span -->
<div> Elemento em nível de Bloco
    Agrupa uma região ou conjunto de elementos.
<span> Elemento em Linha (Inline)
    Agrupa um trecho específico dentro de um texto.
```

Organização e Semântica:

Enquanto a **div** quebra a linha para criar uma nova seção de bloco, o **span** mantém o fluxo do texto intacto, alterando apenas a parte selecionada.

Utilize o `<span>` sempre que precisar aplicar estilos específicos a partes de um texto sem alterar a estrutura do parágrafo.

Quando NÃO usar `<div>`

Evite o uso excessivo de contêineres genéricos e prefira a semântica do HTML5.

Não recomendado (Estrutura genérica)

```
<!-- Divs com classes explicativas -->
<div class="cabecalho">...</div>
<div class="navegacao">...</div>
<div class="conteudo">...</div>
<div class="rodape">...</div>
```

Recomendado (Estrutura semântica)

```
<!-- Tags estruturais semânticas -->
<header>...</header>
<nav>...</nav>
<main>...</main>
<footer>...</footer>
```

Regra prática: Se existe uma tag HTML que descreve semanticamente o conteúdo, prefira utilizá-la. Use `div` apenas como um contêiner genérico quando não houver outra alternativa.

Atributos

Elementos podem possuir atributos para configurar ou fornecer informações adicionais.

Link com Atributo único

```
<!-- Tag de Link -->
<a href="/products">
 Produtos
</a>
```

Atributo `href`:
Especifica o destino do link (URL ou caminho interno) para onde o usuário será direcionado ao clicar no elemento.

Imagem com Múltiplos Atributos

```
<!-- Tag de Imagem -->
<img
 src="foto.jpg"
 alt="Notebook"
>
```

Atributos `src` e `alt`:
O atributo **src** define a origem do arquivo de imagem, enquanto o **alt** fornece uma descrição de texto alternativo.

Atributos modificam o comportamento padrão de uma tag e são declarados sempre na tag de abertura.

Atributos `id` e `class`

Dois recursos fundamentais do HTML para identificar, estilizar e manipular elementos na página.

O Atributo id (Identificador Único)

```
<!-- Identifica um elemento único -->
<div id="menu">
  Conteúdo do menu...
</div>
```

Identificação Exclusiva:
Diferente de outros atributos, o `id` é exclusivo. Ele deve identificar apenas um único elemento específico em todo o documento HTML.

O Atributo class (Estilo Reutilizável)

```
<!-- Associa elementos a um grupo -->
<div class="card">
  .card { color: blue; }
</div>
```

Associação de Grupo:
Permite aplicar o mesmo estilo ou comportamento a múltiplos elementos. Use para agrupar padrões visuais reutilizáveis.

Além do CSS, scripts JavaScript também utilizam frequentemente estes identificadores (`id` e `class`) para interagir dinamicamente com a página.

HTML + CSS + JavaScript

Como as três tecnologias trabalham juntas dividindo responsabilidades na Web.

HTML

Estrutura / Conteúdo

```
<!-- Marcação -->
<button>Comprar</button>
```

Define e organiza os elementos, textos, botões e a estrutura fundamental da página.

CSS

Aparência / Estilo

```
/* Estilização */
button {
  color: blue;
}
```

Responsável por cores, fontes, espaçamentos, layout visual e toda a parte estética.

JavaScript

Comportamento

```
// Interatividade
btn.onclick = () => {
  comprar();
}
```

Adiciona dinamismo, lógica e responde às interações do usuário, como cliques e requisições.

Cada tecnologia cumpre um papel único e essencial para uma experiência web completa.

O navegador interpreta HTML

Ao acessar `https://example.com`, a comunicação e a renderização seguem este fluxo:

1. Requisição



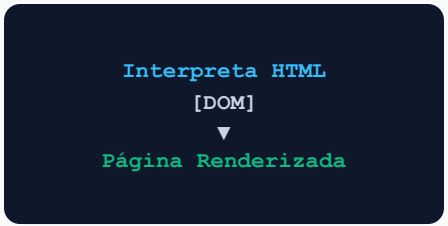
O usuário solicita o endereço e o navegador envia um pedido HTTP GET ao servidor remoto.

2. Resposta



O servidor processa a chamada e devolve o documento HTML de resposta para o cliente.

3. Renderização



O navegador interpreta o código bruto, constrói a árvore DOM e exibe a página de forma legível.

Todo esse ciclo ocorre em milissegundos para garantir uma navegação fluida ao usuário.

HTML e DOM

O navegador transforma o HTML em uma estrutura chamada **DOM (Document Object Model)**.

Documento HTML

```
<body>
  <h1>Produtos</h1>
  <p>Lista de produtos</p>
</body>
```

Estrutura Conceitual do DOM

```
body
├── h1
│    └── "Produtos"
│
└── p
     └── "Lista de produtos"
```

Essa estrutura será muito importante quando estudarmos JavaScript.

Inspecionando HTML

Utilize o DevTools do navegador para analisar e compreender a estrutura técnica de qualquer página.

Como Acessar (DevTools)

```
<!-- Atalhos comuns no teclado -->
Pressione F12 ou Ctrl+Shift+I
Navegar para a aba "Elements"
```

Inspeção Direta:
O painel se abrirá integrado ao navegador, permitindo a visualização instantânea de todos os elementos estruturais do documento.

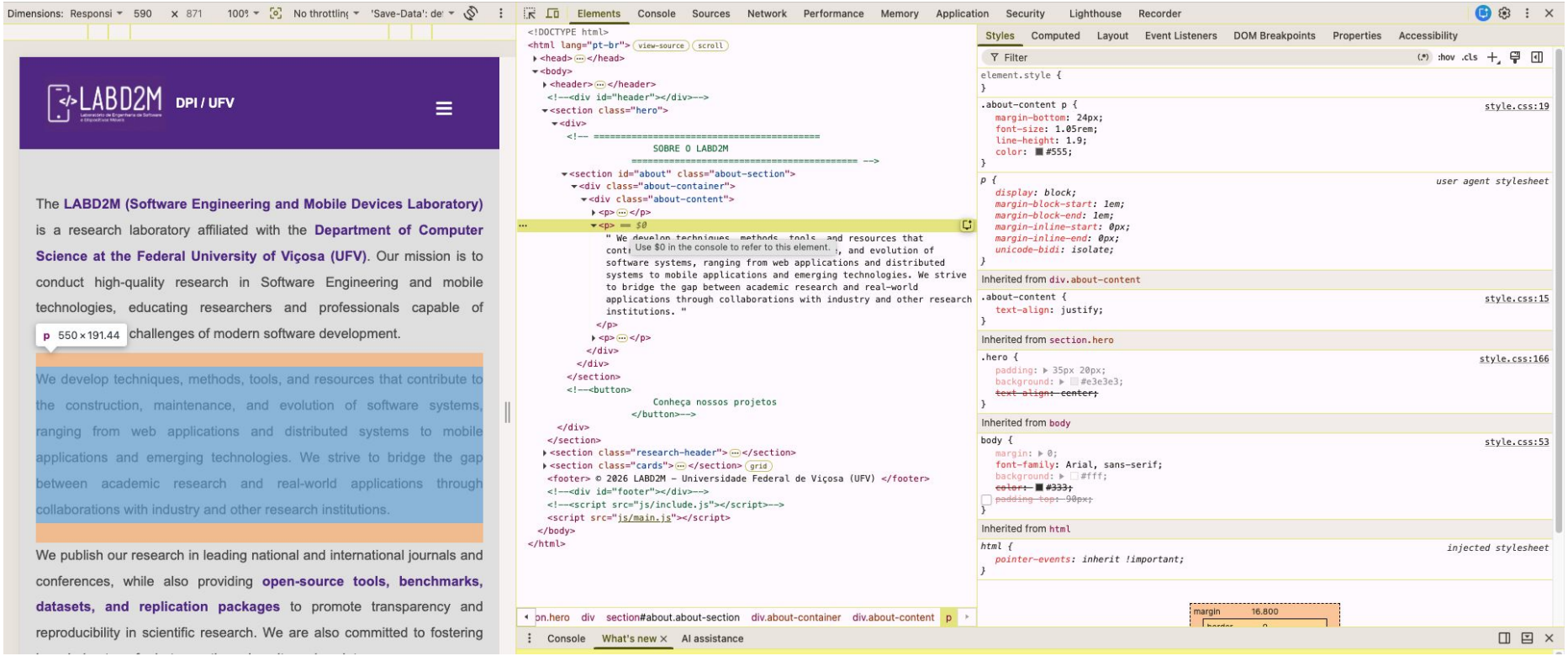
O que podemos observar

```
<!-- Elementos inspecionados -->
• Código HTML bruto
• Atributos, classes e estilo CSS
```

Análise Completa:
Explore e modifique de forma interativa os elementos, classes, ID e a própria estrutura hierárquica do DOM em tempo real.

Experimente: Clique com o botão direito em qualquer página → Inspecionar.

Inspecionando HTML



Editando HTML no Navegador

Altere o código de qualquer página diretamente no DevTools para entender o DOM local:

Atividade Prática

1. Abra uma página qualquer.
2. Pressione F12 (DevTools) no teclado.
3. Vá até a aba Elements.
4. Encontre uma tag <h1>.
5. Dê dois cliques sobre o texto.
6. Altere para "Olá, turma!"

Análise do Resultado

A alteração modificou o site no servidor?

NÃO!

A mudança que você fez ocorreu estritamente no `DOM local` do seu navegador.

Nenhum outro usuário acessando o site verá essa alteração, pois os arquivos originais no servidor continuam intactos.

Ao recarregar a página (F5), o navegador solicita novamente o HTML ao servidor e reconstrói o DOM original.

Demonstração Prática

Criando nossa primeira página

Demonstração Prática

Siga os passos abaixo:

1. Crie um arquivo chamado `index.html` no seu computador.
2. Copie e cole a estrutura do código HTML exibida no painel ao lado.
3. Salve o arquivo.
4. Dê um duplo clique no arquivo para abri-lo no seu navegador favorito.

```html
<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8">
    <title>Minha primeira página</title>
  </head>
  <body>
    <h1>Minha primeira página Web</h1>
    <p>
      Estou aprendendo desenvolvimento Web.
    </p>
  </body>
</html>
```

Prontinho! O navegador interpretará essas tags e renderizará sua primeira página web local.

Evoluindo a página

Agora adicione os seguintes elementos ao seu código HTML:

Um link

```
<a href="https://www.ufv.br">
  Universidade Federal de
  Viçosa
</a>
```

Uma imagem

```
<img
  src="imagem.jpg"
  alt="Descrição..."
>
```

Uma lista

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

HTML correto × HTML que funciona

Uma página pode "funcionar" e ainda estar mal estruturada de forma não-semântica.

Apenas Funciona (Não Semântico)

```
<div>
  <div>Produtos</div>
  <div>Lista de produtos</div>
</div>
```

Dificulta a acessibilidade, SEO e a manutenção do código a longo prazo.

HTML Correto (Semântico)

```
<header>
  <h1>Produtos</h1>
</header>
<main>
  <section>
    <h2>Lista de produtos</h2>
  </section>
</main>
```

Usa as tags adequadas, facilitando a leitura por motores de busca e leitores de tela.

O objetivo não é apenas "aparecer no navegador". É produzir HTML estruturado, semântico e acessível.

Boas práticas de HTML

HTML semântico

Use as tags corretas para indicar o significado do conteúdo:

```
<header>
<nav>
<main>
<section>
<article>
<footer>
```

Alt em imagens

Sempre use descrição alternativa para garantir acessibilidade:

```
<img
src="..."
alt="..."
```

Associe labels

Vincule rótulos aos campos de entrada de forma explícita:

```
<label for="em">
Email
</label>
<input
id="em"
type="email"
```

Hierarquia

Respeite a ordem lógica e estrutural dos cabeçalhos:

```
<h1>Título 1
└── <h2>Sub
    └── <h3>Tópico
```

O que evitar?

Não faça

```
<br><br><br><br>
```

Evite usar quebras de linha múltiplas e repetidas para forçar ou criar espaçamento vertical na página.

Não use

```
<center>
```

Não utilize esta tag obsoleta para centralizar elementos. Ela foi totalmente descontinuada no padrão HTML moderno.

Não use

```
<font>
```

Evite o uso de tags de fonte legadas para controlar cores, tamanhos, fontes ou qualquer aspecto de apresentação do texto.

Todas essas responsabilidades de layout, espaçamento e aparência pertencem ao CSS.

HTML é estrutura

Se a pergunta é **"o que é isso?"**, pense em HTML.

Identificando Elementos

- Isso é um título? `HTML`
- Isso é um parágrafo? `HTML`
- Isso é uma imagem? `HTML`
- Isso é um link? `HTML`
- Isso é uma lista? `HTML`

E depois da estrutura?

"Como isso deve aparecer?"
`CSS`

"Como isso deve se comportar?"
`JavaScript`

O HTML define a semântica e os blocos de construção da página. O estilo visual (CSS) e as ações (JavaScript) vêm sempre depois.

HTML e nossa arquitetura Web

Retomando a arquitetura cliente-servidor:

Front-End

Interface e Interação

`HTML + CSS + JavaScript`

HTTP

Back-End

Regras de Negócio e Dados

`Elixir + Phoenix`

Foco de Hoje: `HTML` Compreender a estrutura semântica essencial da Web.

Próxima Etapa: `CSS` Aprender a tornar essa estrutura visualmente agradável e profissional.

Exercício de fixação

Qual elemento HTML você utilizaria para cada situação?

- Título principal < ? >
- Parágrafo < ? >
- Link < ? >
- Imagem < ? >
- Lista não ordenada < ? >
- Item de lista < ? >
- Tabela < ? >
- Campo de texto < ? >
- Botão < ? >
- Formulário < ? >
- Conteúdo principal < ? >
- Navegação < ? >

Exercício de interpretação

Considere o código HTML abaixo para responder às questões propostas:

```html
<!-- Código HTML -->
<a href="/products">
   <img
      src="notebook.jpg"
      alt="Notebook"
   >
   <h2>Notebook</h2>
   <p>R$ 5.000</p>
</a>
```

1. Qual é o elemento principal?
2. Qual é o atributo do link?
3. Qual é o atributo da imagem que indica seu arquivo?
4. Qual atributo descreve a imagem?
5. Qual é o título do produto?
6. Para onde o usuário será direcionado ao clicar?

Atividades Práticas

Desafio Final - Parte 1

Construa uma página Web para uma disciplina da UFV.

O que a página deve conter:

- ✓ Título da disciplina
- ✓ Descrição e Professor
- ✓ Conteúdo e Links úteis
- ✓ Imagem e Lista de tópicos
- ✓ Tabela de horários
- ✓ Formulário de contato

Requisitos (Tags obrigatórias):

`<h1> <h2> <p> <a> <img>`

`<ul>` ou `<ol> <table>`

`<form> <input> <button>`

`<header> <main>`

`<section> <footer>`

Desafio Final - Parte 2: Criando uma página de produtos

Atividade 2: Desafio Prático

Crie uma página de produtos que contenha:

- Título principal
- Descrição da página
- **Pelo menos 3 produtos**, cada um com:
  - Imagem (foto);
  - Nome do produto;
  - Preço formatado;
  - Link para detalhes
- Formulário funcional para cadastrar produto



Neste momento, não se preocupe com aparência. O foco principal é a estrutura HTML!

Desafio Final - Parte 2: Estrutura sugerida para a atividade

Organização Semântica

<header> Define o topo da página, ideal para o título principal ou logomarca.

<main> Concentra o conteúdo principal do fluxo de navegação da página.

<section> Agrupa e divide logicamente as seções de produtos e de cadastro.

<footer> Rodapé contendo informações institucionais ou direitos autorais.

```
<body>
  <header>
    <h1>Produtos</h1>
  </header>
  <main>
    <section>
      <h2>Produtos disponíveis</h2>
      <!-- produtos -->
    </section>
    <section>
      <h2>Cadastrar produto</h2>
      <!-- formulário -->
    </section>
  </main>
  <footer>
    <p>Minha loja</p>
  </footer>
</body>
```

Agora é com vocês…



Bons estudos!

Prof. Lucas Vegi